

Divide and Conquer

ATONU ROY CHOWDHURY, atonuroy.chowdhury@bracu.ac.bd

July 3, 2026

Divide and conquer is a widely used and an extremely important problem solving tactic. Here, what we essentially do is: we have a difficult problem, which we don't know how to solve yet. We then divide the problem into multiple subproblems, which might be easier to solve. Then we solve the subproblems, and then combine the solutions of the subproblems.

A divide and conquer algorithm lets us forget the details of solving the subproblems, and we have to only worry about combining the solutions of the subproblems to form the main solution.

For divide and conquer algorithms, while analyzing the time complexity, we oftentimes use the Master theorem. Let's quickly recall what it says.

Theorem 1 (Theorem 4.1 from [1], also known as the Master Theorem)

If an algorithm takes $T(n)$ time, and it can be written recursively as

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n),$$

then

1. If there exists a constant $\epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$, then

$$T(n) = \Theta(n^{\log_b a}).$$

2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} (\lg n)^k)$, then

$$T(n) = \Theta(n^{\log_b a} (\lg n)^{k+1}).$$

3. If there exists a constant $\epsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \epsilon})$, and if $f(n)$ additionally satisfies the **regularity condition** $a f(n/b) \leq c f(n)$ for some constant $c < 1$ and all sufficiently large n , then

$$T(n) = \Theta(f(n)).$$

To summarize, we compare between $n^{\log_b a}$ and $f(n)$, and whichever of these two are **asymptotically larger**, that one dominates, and that one becomes $T(n)$. If they are the same, i.e. when $f(n) = n^{\log_b a}$ (asymptotically), then $T(n) = \Theta(f(n) \lg n) = \Theta(n^{\log_b a} \lg n)$.

In case of divide and conquer algorithms, let's see how to interpret the quantities. In the recursive equation,

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

it essentially means that

- we have divided our n -sized problem into $\frac{n}{b}$ -sized subproblems, and the number of subproblems is a ;
- $T\left(\frac{n}{b}\right)$ is the time it takes to solve one subproblem;
- $f(n)$ is the time it takes while dividing into subproblems and then later combining the solution of the subproblems.

§1 Counting inversions

Problem 1. You're given an array `arr` of size n , you have to sort it and count the number of *inversions* in the array.

An inversion is a pair (i, j) with $i < j$ and `arr[i] > arr[j]`.

An inversion is basically when a pair of elements are not in (ascending) sorted order. When the array is sorted, the number of inversions is 0. On the other hand, when the array is reversely sorted (i.e. sorted in descending order) then the inversion count is highest. In that case, every pair is an inversion. So the inversion count will be

$$\binom{n}{2} = \frac{n(n-1)}{2}.$$

So we can say that the inversion count is a measure of how far the array is from being sorted. If the inversion count is low, we can deduce that the array is pretty close to being sorted. On the contrary, if the inversion count is very large, we can infer that the array is far from being sorted. Let's see an example.

1	5	7	9	3	2
0	1	2	3	4	5

Here (1,4) is an inversion, because the elements in position 1 and 4 are out of order, i.e. `arr[1] > arr[4]`. We have in total 7 inversions in this array, which are highlighted below.

1	5	7	9	3	2
0	1	2	3	4	5
1	5	7	9	3	2
0	1	2	3	4	5
1	5	7	9	3	2
0	1	2	3	4	5
1	5	7	9	3	2
0	1	2	3	4	5

1	5	7	9	3	2
0	1	2	3	4	5

1	5	7	9	3	2
0	1	2	3	4	5

Now, how do we count the number of inversions here? The obvious solution is iterating through all the pairs, and check if there is an inversion.

```

1 function countInversion(arr)
2     count = 0
3     for i = 0, 1, ... , n-1 do
4         for j = i+1, ... , n-1 do
5             if arr[i] > arr[j] then
6                 count++
7
8     return count

```

This is a very simple algorithm, which will give us the correct answer clearly. But its time complexity is $O(n^2)$. Can we do better?

Let's try a divide and conquer approach. Suppose we have divided our array `arr` into two halves, say `left` and `right` to denote the left half and the right half, respectively. An inversion (i, j) can appear in 3 different cases:

1. both i and j are in `left`;
2. both i and j are in `right`;
3. i is in `left`, but j is in `right`.

When we solve the subproblem for `left` and `right`, the first two cases are already solved. For this purpose, we'll recursively call the main function on `left` and `right`. Now the third case is what's left.

Let's call the third type of inversion "*cross-inversion*". We can count the number of cross-inversions by iterating through the `left` and `right` array, in the following way:

```

1 function countInversion(arr)
2     left = arr[0, ... , n/2]
3     right = arr[n/2+1, ... , n-1]
4
5     leftInv = countInversion(left)
6     rightInv = countInversion(right)
7     crossInv = 0
8
9     for x in left do
10         for y in right do
11             if x > y then
12                 crossInv ++
13
14     return leftInv + rightInv + crossInv

```

Now, to analyze the time complexity of this approach, we are dividing the n -sized problem into two $\frac{n}{2}$ -sized subproblems. Finally, in the combine step, we are iterating through both the arrays `left` and `right`. It takes $\frac{n}{2} \cdot \frac{n}{2} = \frac{n^2}{4}$ time. So, the overall time complexity is

$$T(n) = 2 T\left(\frac{n}{2}\right) + \frac{n^2}{4}.$$

Here, $n^{\log_b a} = n$, which is *asymptotically* smaller than $f(n) = \Theta(n^2)$. Therefore, the time complexity is $T(n) = \Theta(f(n)) = \Theta(n^2)$.

Although we have used a divide and conquer approach here, but the time complexity doesn't improve. This is because we are still iterating through the whole array, in a sense. Also, we are incrementing the inversion count by 1, at most. The maximum value of the inversion is $\frac{n^2-n}{2}$. Therefore, in the worst case, i.e. in case when the inversion count is highest, the algorithm will do up to $\frac{n^2-n}{2}$ computations. That's why the complexity is $\Theta(n^2)$.

How can we do better? We can adopt a merge-sort style approach. At each step, along with the inversion count, we will also sort the arrays. In that case, the `left` and the `right` arrays will be sorted when we're counting the cross inversions. How does this help us? Consider the previous example:

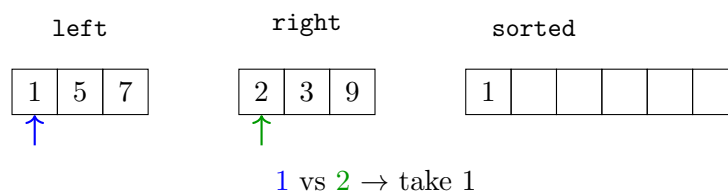
1	5	7	9	3	2
---	---	---	---	---	---

Suppose the `left` subarray and the `right` subarray are both sorted:

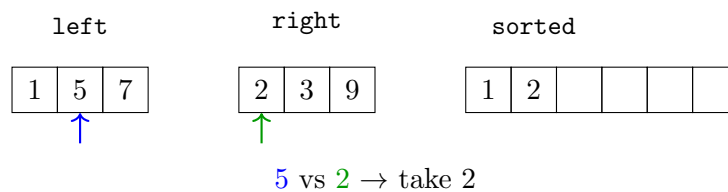
1	5	7
---	---	---

2	3	9
---	---	---

Now while doing merge-sort, we'll keep track of the cross-inversions.



Here, the left element is smaller, so there are no cross inversions.



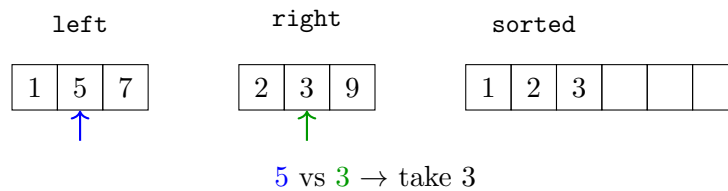
Now, here, the right element is smaller. Therefore, there is an inversion. The elements (5,2) produce an inversion here. So we keep 2 in the sorted array and move the green pointer.

But notice that, all the elements after 5 are all larger than 5 (since the `left` array is sorted). Therefore, they also produce an inversion with 2. For example, in the above scenario, the elements (7,2) produce an inversion. But as we move the green pointer, we can't keep track of that inversion. Hence, we can't increment the `crossInv` counter by 1. We must increase it by the number of elements remaining in the `left` subarray, **starting from the pointer position**.

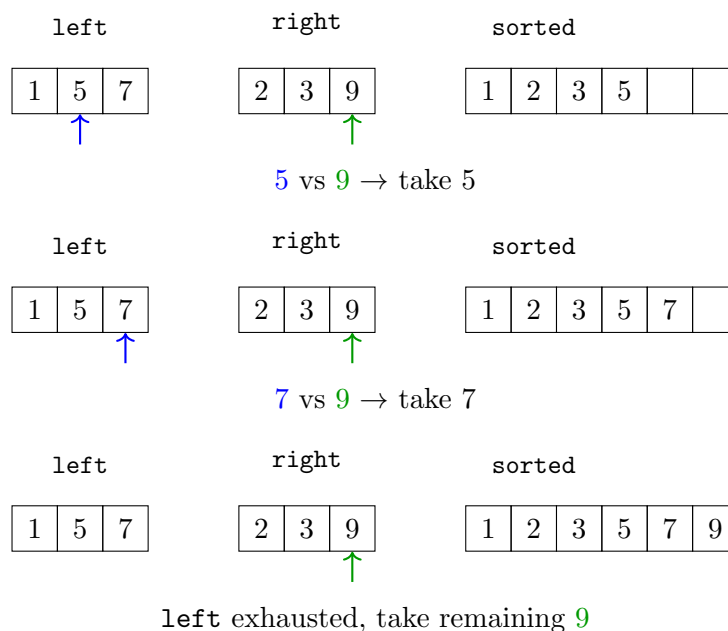
If the left-pointer is in index `i` of the `left` subarray, then the number of elements remaining in the `left` subarray starting from the pointer position is `len(left) - i`. Therefore, we have to do we must have

$$\text{crossInv} = \text{crossInv} + (\text{len}(\text{left}) - i).$$

In the following step:



The right element is smaller. Therefore, there is an inversion. The elements (5,3) produce an inversion here. So we keep 3 in the sorted array, increase `crossInv` by `len(left) - i` and move the green pointer. We keep repeating this until we're done.



No further cross-inversions are found. Therefore, in this case, the number of cross inversions is 4. So the final algorithm looks like this:

```

1 function countInversion(arr) // returns the inversion count and the sorted array
2   if len(arr) <= 1 then
3     return 0, arr
4
5   mid = len(arr) / 2
6   left = arr[0, ... , mid]
7   right = arr[mid+1, ... , n-1]
8
9   leftInv, left = countInversion(left) // now left is sorted
10  rightInv, right = countInversion(right) // now right is sorted
11  crossInv, sorted = merge(left, right)
12
13  return leftInv + rightInv + crossInv, sorted
14
15 function merge(left, right) // returns cross-inversion count and merge-sorted array
16   crossInv = 0, i = 0, j = 0
17
18   while i < len(left) and j < len(right) do
19     if left[i] <= right[j] then
20       sorted.add(left[i])
21       i++
22
23     else if left[i] > right[j] then
24       sorted.add(right[j])
25       crossInv += (len(left) - i)
26       j++
27
28   sorted.add(left[i, ... ,])
29   sorted.add(right[j, ... ,])
30
31   return crossInv, sorted

```

Now, what about the time complexity of this algorithm? The `merge` function goes through all the elements of `left` and `right` exactly once. So it runs in linear time, i.e. $O(n)$. Therefore, the recursive equation becomes:

$$T(n) = 2 T\left(\frac{n}{2}\right) + n.$$

Since $f(n) = n = n^{\log_b a}$, by Master theorem,

$$T(n) = \Theta(n \lg n).$$

§2 Counting pseudo-inversions

Problem 2. You're given an array `arr` of size n , you have find the number of pairs of indices (i, j) with $i < j$ and $\text{arr}[i] > \text{arr}[j]^2$.

Note that this is pretty similar to the previous question. An inversion happens when $i < j$ and $\text{arr}[i] > \text{arr}[j]$. Here, the situation we are dealing with is $i < j$ and $\text{arr}[i] > \text{arr}[j]^2$. Let's call such pairs of indices (i, j) a “pseudo-inversion”.

Now, how do we count pseudo-inversions? An obvious solution is, again, iterating through all possible pairs (i, j) , which will give us an $O(n^2)$ solution. How can we do better? Can we adopt a similar strategy as before?

We can surely try to do divide-and-conquer as before:

```

1 function countPseudoInversion(arr)
2     mid = len(arr)/2
3     left = arr[0, ... , mid]
4     right = arr[mid+1, ... , n-1]
5
6     leftInv = countPseudoInversion(left)
7     rightInv = countPseudoInversion(right)
8     crossInv = 0
9
10    // Then we somehow find the crossInv count here
11
12    return leftInv + rightInv + crossInv

```

The key difficulty lies in finding the count of `crossInv`. For every value x in `left`, we need to find how many value y there are in `right` such that $x > y^2$. Sure, we can do it the obvious way:

```

1 for x in left do
2     for y in right do
3         if x > y * y then
4             crossInv ++

```

This step takes $\frac{n}{2} \cdot \frac{n}{2} = \frac{n^2}{4}$ time. So, the overall time complexity is

$$T(n) = 2 T\left(\frac{n}{2}\right) + \frac{n^2}{4}.$$

Here, $n^{\log_b a} = n$, which is *asymptotically* smaller than $f(n) = \Theta(n^2)$. Therefore, the time complexity is $T(n) = \Theta(f(n)) = \Theta(n^2)$. This is not what we want, we want to do better.

We can try to sort the array along the way, in a merge-sort like way. But in a sorted array, there is no obvious way to compare x with y^2 . Also, a sorted array won't even work, if we want to adopt a merge-sort like approach. Consider the following example where both `left` and `right` are sorted:

2	5	7	10
---	---	---	----

-2	1	9	25
----	---	---	----

Here, the sorting doesn't give us any extra information. Because if we pick the element 2 from **left**, we have $2 \not\geq (-2)^2$, i.e. $2 \leq (-2)^2$. The latter elements of **right** are all larger than -2 . But we can't say that their squares are larger than $(-2)^2$.

If we could make the conclusion, then we could've discarded the latter elements in **right** once we found one y with $x \not\geq y^2$. To facilitate this, we would sort the arrays **based on absolute value**. This is because

$$\text{if } x \leq y^2 \text{ and } |y| \leq |z|, \text{ then } x \leq z^2.$$

If z appears after y (which would happen if $|y| \leq |z|$ and the array is sorted by absolute value), then once we found one y with $x \not\geq y^2$, we would also have $x \not\geq z^2$. So z can be discarded automatically.

For example, if **left** and **right** are both sorted by absolute value (or let's call them *abs-sorted*) in the previous example,

2	5	7	10	1	-2	9	25
---	---	---	----	---	----	---	----

$$2 > 1^2, \quad \text{and} \quad 2 \not\geq (-2)^2,$$

and now we can simply ignore the terms after -2 in **right**.

Therefore, after abs-sorting both **left** and **right**, for every x in **left**, we have to find how many y there are in **right** such that $x > y^2$. Suppose the **merge** function takes care of this as well, on top of merge-sorting **left** and **right** based on absolute value.

```

1 function countPseudoInversion(arr) // returns the pseudo-inversion count and the
   abs-sorted array
2   if len(arr) <= 1 then
3     return 0, arr
4
5   mid = len(arr) / 2
6   left = arr[0, ... , mid]
7   right = arr[mid+1, ... , n-1]
8
9   leftInv, left = countInversion(left) // now left is abs-sorted
10  rightInv, right = countInversion(right) // now right is abs-sorted
11  crossInv, sorted = merge(left, right)
12
13  return leftInv + rightInv + crossInv, sorted

```

Now, how can we design the **merge** function? Of course it will merge-sort **left** and **right** based on absolute value. On top of that it needs to calculate the **crossInv** counts.

For every x in **left**, we will find how many y in **right** there are such that $x > y^2$. Note that, this number is equal to the index of the **first** y in **right** such that $x \not\geq y^2$. We will find the **first** such y using binary search.

For a given x in `left`, we are now searching for the first y with $x \not\geq y^2$. For binary search, our initial search space is the whole array. Then we take the midpoint, and compare x with the square of the midpoint.

- If $x > \text{right}[\text{mid}]^2$, then our desired y with $x \not\geq y^2$ lies in the right half of our search space.
- If $x \leq \text{right}[\text{mid}]^2$, then our desired y with $x \not\geq y^2$ lies in the left half of our search space.

Therefore, the binary search will look like this:

```

1 lo = 0, hi = len(right)
2   while lo < hi do
3       mid = (lo + hi) / 2
4       if x > right[mid] * right[mid] then
5           lo = mid + 1
6       else
7           hi = mid
8   crossInv += lo

```

Finally, after combining everything, the `merge` function will be:

```

1 function merge(left, right) // returns cross-pseudo-inversion count and abs-sorted
   array
2   crossInv = 0, i = 0, j = 0
3
4   while i < len(left) and j < len(right) do
5       if abs(left[i]) <= abs(right[j]) then
6           sorted.add(left[i])
7           i++
8
9       else
10          sorted.add(right[j])
11          j++
12
13   sorted.add(left[i, ... ,])
14   sorted.add(right[j, ... ,])
15
16   for x in left do
17       lo = 0, hi = len(right)
18       while lo < hi do
19           mid = (lo + hi) / 2
20           if x > right[mid] ** 2 then
21               lo = mid + 1
22           else
23               hi = mid
24       crossInv += lo
25
26   return crossInv, sorted

```

Now, what about the time complexity of this algorithm? The `merge` function first merge-sorts `left` and `right`, which takes $O(n)$ time. Then it performs a binary search for every element of `left`. There are $\frac{n}{2}$ binary searches going on here, and each of those searches has a search-space length of $\frac{n}{2}$. Therefore, the `merge` function has a time complexity of

$$\Theta\left(n + \frac{n}{2} \lg\left(\frac{n}{2}\right)\right) = \Theta(n \lg n).$$

Therefore, the recursive equation becomes:

$$T(n) = 2 T\left(\frac{n}{2}\right) + \Theta(n \lg n) ..$$

Since $f(n) = \Theta(n \lg n) = \Theta(n^{\log_b a} \lg n)$, by Master theorem,

$$T(n) = \Theta\left(n (\lg n)^2\right).$$

§3 Find powers faster

Problem 3. You are given two integers a , b , and m . Calculate $a^b \bmod m$.

When we analyze complexities, we usually ignore the time taken by arithmetic operations. This is because for small numbers, addition, multiplication, subtraction, division doesn't take much time. But for large numbers, multiplication is computationally very heavy. If we want to multiply two n -digit numbers,

- the naive approach that we use everyday takes $O(n^2)$ time;
- Karatsuba's algorithm takes $O(n^{\lg 3}) = O(n^{1.585...})$ time;
- using Fast Fourier Transform (FFT), we can speed it up to $O(n \lg n)$.

So we understand that multiplying large numbers is a heavy task. And here, as we keep multiplying numbers to get to a^b ,

$$a^b = a \times a \times a \times \cdots \times a,$$

the numbers get very large. As a result, the multiplication takes a lot of time.

Instead, we will adopt a divide and conquer approach. How so? To find $a^b \bmod m$, we will first find $a^{b/2} \bmod m$. Then we'll square that. This will make our problem much simpler.

Now $a^{b/2} \bmod m$ is a number smaller than m . So squaring it will not take much time, since m is a constant.

We have a final caveat. If b is even, squaring $a^{b/2} \bmod m$ is sufficient. But if b is odd, then we need to multiply a with the squared value. Therefore, the final algorithm becomes:

```

1 function power(a, b, m)
2     if b == 0 return 1
3
4     half = power(a, b/2, m)
5     res = (half * half) % m
6
7     if b % 2 == 1 then
8         res = (res * a) % m
9
10    return res

```

Now, for this problem, solving a b -sized problem entails solving a $\frac{b}{2}$ -sized problem once (because we are calling `power(a, b/2, m)` only once). Then the rest of the operations are multiplication and mod by relatively small numbers, which can be considered $O(1)$ time. Therefore, the recursive equation for time complexity becomes

$$T(n) = T\left(\frac{n}{2}\right) + 1.$$

Here, $n^{\log_b a} = n^{\log_2 1} = n^0 = 1 = f(n)$, therefore,

$$T(n) = \Theta(f(n) \lg n) = \Theta(\lg n).$$

Problem 4. You are given a 2×2 integer matrix A and an integer exponent n . Your task is to compute the matrix A^n , where all intermediate operations (additions and multiplications during matrix multiplication) are performed modulo $10^9 + 7$.

This follows exactly the same algorithm as the previous example. For even n ,

$$A^n = A^{n/2} \cdot A^{n/2}.$$

So we call the function recursively on $n/2$ to find $A^{n/2}$, and then we square it. For odd n , we need to be cautious, similarly as before. After squaring, we have to multiply by A .

Remark 3.1 (A suggestion on implementation). It's better to create a class `Matrix` for 2×2 matrices. Because if you use `long[2][2]` for 2×2 matrices, that's actually 3 objects:

- one `long[][]` object,
- two `long[]` rows.

For each matrix multiplication, if you keep creating new `long[2][2]` objects, you're doing a huge number of small array allocations. This may lead to TLE and MLE, which is why creating a class `Matrix` is recommended.

Problem 5. You are given three integers a , n , and m . Calculate

$$(a^1 + a^2 + a^3 + \cdots + a^n) \mod m.$$

Again, we have a similar issue: multiplying large numbers is computationally very heavy. On top of that, we have to add them. We already know how to find $a^b \mod m$ efficiently. So one thing that we can do is we find $a^b \mod m$ for $b = 1, 2, \dots, n$ and then add them all.

```

1 function powSum(a, n, m)
2     sum = 0
3     for i = 1, 2, ..., n do
4         sum += power(a, i, m) // the power function from problem 3
5
6     return sum

```

That would take $O(n \lg n)$ time, since the outer loop runs n -times, and for each iteration, finding `power(a, i, m)` takes up to $\lg n$ time. But for very large n , this is not very efficient.

The next idea might be to use the geometric series formula:

$$a^1 + a^2 + \cdots + a^n = \frac{a^{n+1} - a}{a - 1}.$$

But the issue is, since this answer is very large, we have to output it in mod m . We can't always divide in modular arithmetic. We can divide by $a - 1$ in mod m if and only if

$$\gcd(a - 1, m) = 1.$$

But that condition is not always guaranteed here. So we can't just apply the formula of geometric series for this problem.

Therefore, we will now devise a divide-and-conquer method to solve this problem. Notice that

$$a^1 + a^2 + \cdots + a^n = (a^1 + a^2 + \cdots + a^{n/2}) + a^{n/2} (a^1 + a^2 + \cdots + a^{n/2}).$$

A divide-and-conquer approach can be applied because all the quantities here are already known by us:

$$\underbrace{a^1 + a^2 + a^3 + \cdots + a^n}_{\text{powSum}(a, n, m)} = \underbrace{(a^1 + a^2 + \cdots + a^{n/2})}_{\text{powSum}(a, n/2, m)} + \underbrace{a^{n/2}}_{\text{power}(a, n/2, m)} \underbrace{(a^1 + a^2 + \cdots + a^{n/2})}_{\text{powSum}(a, n/2, m)}.$$

That means, if we solve the problem for $n/2$, we can use that to solve the problem for n .

For odd n , we just solve the problem for $n - 1$, and then we add $a^n \mod m$. Therefore, the algorithm becomes:

```

1 function powSum(a, n, m)
2   if n == 1 return a
3
4   if n % 2 == 0 then
5     half = powSum(a, n/2, m)
6     return (half + (power(a, n/2, m) * half) % m) % m
7
8   if n % 2 == 1 then
9     return power(a, n, m) + powSum(a, n-1, m)

```

Here, to solve an n -sized problem, we are first solving an $\frac{n}{2}$ -sized problem, and then we are invoking `power(a, n/2, m)`, which takes $\lg n$ time. Therefore,

$$T(n) = T\left(\frac{n}{2}\right) + \lg n.$$

Since $f(n) = \lg n = n^{\log_2 1} \lg n = n^{\log_b a} \lg n$, by Master theorem,

$$T(n) = \Theta\left((\lg n)^2\right).$$

But in reality, we can make it more efficient. Instead of calling the function `power(a, n/2, m)`, we can change the implementation of `powSum(a, n, m)` such that it returns both $a^n \bmod m$ and $(a^1 + \dots + a^n) \bmod m$.

```

1 function powSum(a, n, m) // it returns both sum mod m, and a^n mod m
2   if n == 1 return a, a
3
4   if n % 2 == 0 then
5     half, po = powSum(a, n/2, m)           // half = a^1 + ... + a^{n/2} mod m,
6                                           // po = a^{n/2} mod m
7     res = (half + (po * half) % m) % m
8     an = (po * po) % m
9     return res, an
10
11  if n % 2 == 1
12    sum, po = powSum(a, n-1, m)           // sum = a^1 + ... + a^{n-1} mod m,
13                                           // po = a^{n-1} mod m
14    an = (po * a) % m                     // a^n = a * a^{n-1}
15    res = (sum + an) % m                  // a^n + sum upto a^{n-1}
16    return res, an

```

Now, in this algorithm, for even n ,

$$T(n) = T\left(\frac{n}{2}\right) + O(1),$$

because the other operations deal with very small numbers. For odd n ,

$$T(n) = T(n-1) + O(1) = T\left(\frac{n-1}{2}\right) + O(1).$$

Therefore, overall,

$$T(n) = T\left(\frac{n}{2}\right) + O(1),$$

Here, $n^{\log_b a} = n^{\log_2 1} = n^0 = 1 = f(n)$, therefore,

$$T(n) = \Theta(f(n) \lg n) = \Theta(\lg n).$$

§4 Practice problems

Problem 1. You are given an array `arr` of size N in increasing order. Find an order of these N integers such that, if these integers are inserted into a Binary Search Tree (BST) one by one, the height of the resulting BST is minimized.

A Binary Search Tree is a binary tree in which each node has at most two children, referred to as the left and right child. For any node, all elements in the left subtree are smaller than the node's value, and all elements in the right subtree are greater than the node's value.

The height of a Binary Search Tree is defined as the maximum depth among all the nodes in the tree.

Problem 2. There is a Binary Tree with N nodes. You are given the in-order and pre-order traversals of the tree. Your task is to determine the post-order traversal of the tree.

Problem 3. There is a Binary Tree with N nodes. You are given the in-order and post-order traversals of the tree. Your task is to determine the pre-order traversal of the tree.

References

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th ed., MIT Press, Cambridge, MA, 2022.